

4. Python

Thursday, June 12, 2025 8:43 AM

Why Python for Data Engineers?

- **Versatile:** Works with files, databases, APIs, cloud, big data.
- **Simple:** Easy-to-read syntax, great for beginners and professionals.
- **Powerful Libraries:** Pandas, NumPy, PySpark, SQLAlchemy, Airflow.
- **Community Support:** Huge ecosystem with tons of documentation.
- **Platform-independent:** Works on Linux, Windows, Mac, Cloud.

Python has become the #1 language for data engineering.

Python Setup, Basic Concepts & Virtual Environments

4.1 Python Setup

Check if Python is installed:

```
python3 --version
```

Install Python (if not installed):

- **On Ubuntu/Linux:**
`sudo apt update`

`sudo apt install python3 python3-pip`
- **On Windows/Mac:**
Download installer from:
<https://www.python.org/downloads/>

Install `pip` (Python package manager)

- Usually comes pre-installed.
- Check:
`pip3 --version`

4.2 — IDE (Editor) Setup

IDE	Why Use It
VSCode	Lightweight, powerful extensions
Jupyter Lab	Great for data analysis
PyCharm	Full IDE, great for larger projects

4.3 — Running Python Code

Create a new script file:

Create `hello.py`:

```
print("Hello Data Engineer!")
```

Run in terminal:

```
python3 hello.py
```

Try interactive mode:

```
python3
>>> print("Hello!")
>>> 5 + 2
```

4.4 — Virtual Environments

Why use virtual environments?

- Isolate project dependencies.
- Prevent version conflicts.
- Clean, professional development practice.

Create virtual environment:

```
python3 -m venv venv
```

Activate virtual environment:

- On Linux/Mac:
`source venv/bin/activate`
- On Windows:
`venv\Scripts\activate`

Deactivate virtual environment:

```
deactivate
```

Install packages inside venv:

```
pip install pandas requests sqlalchemy
```

4.5 — Python Basic Concepts

Variables

- `name = "Data Engineering"`
- `age = 25`
- `salary = 65000.50`
- `is_active = True`

Data Types

Type	Example
Integer	10
Float	10.5
String	"Hello"
Boolean	True / False

Control Flow

If-Else

```
if age > 18:  
    print("Adult")  
else:  
    print("Minor")
```

For Loop

```
for i in range(5):  
    Print(i)
```

While Loop

```
count = 0
```

```
while count < 5:
```

```
    print(count)
```

```
    count += 1
```

Functions

```
def greet(name):  
  
    print(f"Hello, {name}!")
```

```
greet("Data Engineering")
```

Data Collections — Lists, Tuples, Sets, Dictionaries

4.6 Lists

- Ordered collection.
- Can store any data type.
- Mutable (can be changed).

Example:

```
fruits = ["apple", "banana", "cherry"]
```

```
print(fruits)
```

Access elements:

```
print(fruits[0]) # apple
```

```
print(fruits[-1]) # cherry (negative index means from end)
```

Modify list:

```
fruits.append("orange")
```

```
fruits.remove("banana")
```

```
print(fruits)
```

Loop through list:

```
for fruit in fruits:
```

```
    print(fruit)
```

4.7 Tuples

- Ordered collection.
- Immutable (cannot be changed after creation).
- Used for fixed data structures (coordinates, database records, etc.).

Example:

```
coordinates = (10, 20)
```

```
print(coordinates[0])
```

Tuple unpacking:

```
x, y = coordinates
```

```
print(x)
```

```
print(y)
```

4.8 Sets

- Unordered collection.
- Unique elements only.
- Very useful for deduplication.

Example:

```
numbers = {1, 2, 3, 3, 4}
```

```
print(numbers) # Output: {1, 2, 3, 4}
```

Set operations:

```
numbers.add(5)
```

```
numbers.remove(2)
```

```
print(numbers)
```

Deduplication example:

```
data = [1, 2, 3, 2, 1, 4]
```

```
unique_data = set(data)
```

```
print(unique_data)
```

4.9 Dictionaries

- Key-Value pairs.
- Extremely useful for structured data (JSON, API responses, etc.).

Example:

```
person = {
    "name": "Krishna",
    "age": 25,
```

```
    "city": "Chicago"
}
```

```
Access values:
print(person["name"])

print(person.get("age"))
```

```
Modify dictionary:
person["age"] = 26

person["country"] = "USA"
```

```
Loop through dictionary:
for key, value in person.items():

    print(f"{key}: {value}")
```

Collection	Ordered?	Mutable?	Unique?	Example Use Case
List	Yes	Yes	No	Rows of data
Tuple	Yes	No	No	Coordinates, DB keys
Set	No	Yes	Yes	Deduplication
Dictionary	No	Yes	Keys unique	API responses, JSON